

CAMPAIGN OS — NORTH STAR PROMPT

Paste this whole document into Claude Code, or save it as `CLAUDE.md` in the repo root so it's loaded as standing context in every session. This is the anchor — if a build session drifts from this, stop and re-paste it before continuing.

1. What this product actually is

A user pastes their **business website URL**. The system reads the site, understands the business, and drafts a **complete multi-channel campaign** — audience, positioning, email sequence, social/ad copy, SEO plan, GEO plan, and video ad scripts (with actual UGC-style video rendered, not just scripts) — ready for the user to edit. Each channel is powered by a **specialist agent** the user can also open a direct conversation with to refine that piece specifically. Once a campaign runs, the system learns what worked and **proactively tells the user** what to do differently next time — it doesn't wait to be asked.

One line: **paste your website, get a campaign, talk to the agents that built it, and it gets smarter about your business every time you run one.**

2. The brain: Kimi K2.6

All agent reasoning runs on **Kimi K2.6** (Moonshot AI), via an OpenAI-compatible endpoint. Relevant to how you build against it:

- 256K context window — enough to hold a full scraped website + campaign state + agent conversation history in one call without aggressive truncation.
- Native vision **and video input** — it can look at a screenshot of a site or an uploaded product video directly, not just parsed text.
- Native tool/function calling, designed for multi-step agentic workflows — this is the right fit for an orchestrator-plus-specialists pattern, not a fight against the model.
- Has a "thinking" and "instant" mode — use thinking mode for Strategy/Analytics agents, instant mode for fast conversational replies in agent chat, to control latency and cost.

Build one shared `kimi_client` wrapper (auth, retries, JSON-schema validation, thinking-mode toggle) and every agent calls through it. Don't let agents each grow their own API-calling code — that's how this becomes unmaintainable by month two.

3. Core user flow

1. User pastes a website URL
 2. Research Agent scrapes it → structured business understanding
 3. User states a goal ("more bookings," "launch this product," etc.) + budget + t
 4. Orchestrator fans out to: Strategy → Copy/Social → Email → SEO → GEO → Video
 5. User reviews the draft campaign, channel by channel
 6. User edits directly OR opens a chat with that channel's specific agent to refi
 7. User approves and exports/schedules
 8. User logs results (or connects ad accounts later)
 9. Analytics & Learning Agent writes a retrospective, updates business memory, and PROACTIVELY surfaces specific suggestions — not just on request
 10. The next campaign for this business starts smarter because of step 9
-

4. Agent architecture

Build **one reusable agent runtime**, not seven bespoke implementations. An "agent" is: `{ system_prompt, tool_set, memory_scope, output_contract }` run through the shared Kimi client. The difference between the Email Agent and the SEO Agent is configuration, not separate codebases. This is the single most important engineering decision in this document — violate it and you'll be maintaining seven divergent systems within a month.

Orchestrator (Campaign Director) Holds campaign state. On first generation, calls specialist agents in the right order and assembles their outputs into one campaign object. On follow-up, routes a user's chat message to the correct specialist agent based on what they're asking about.

Research Agent Input: a URL. Scrapes the site (reuse the Bright Data setup from the Metalyzi SpareRoom scraper — you already have working infrastructure for this, don't reinvent it). Output: `{ business_summary, offer, price_signals, tone_examples[], products_or_services[], existing_brand_assets[], target_customer_signals[], competitor_mentions[] }`.

Strategy Agent Input: Research Agent output + user goal + budget + timeframe. Output: positioning, audience segments, funnel/channel plan. (This is the old Steps 1–3 from the pipeline version — same job, now framed as one specialist instead of three)

pipeline stages.)

Copy & Social Agent Organic social posts + paid ad copy, per channel, in variants.

Email Agent Nurture sequences, subject line variants, send-timing suggestions.

SEO Agent Keyword targets, on-page recommendations, content briefs for the business's own site.

GEO Agent — distinct from SEO, don't merge them Generative Engine Optimization: structures content so AI answer engines (ChatGPT, Perplexity, Google AI Overviews) cite the business. In 2026 this is graded differently from SEO — it rewards content depth, clear FAQ-style structure, and citation-worthy specificity over keyword density. Deliverables: FAQ blocks answering real customer questions, schema markup snippets, and a short list of "citable claims" the business can publish (stats, original data points) — these compound over time and are worth having as a distinct agent output, not a subsection of SEO.

Video/UGC Agent — build this AFTER the text-based loop works, not alongside it Generates a script, then calls a third-party rendering API to produce an actual video — don't try to render video yourselves. Two real options, both API-accessible:

- **Creatify** — paste a URL/product page, get a video ad back. Best fit for you specifically since you're already scraping the website; least new integration surface.
- **HeyGen** — stronger for avatar-driven, multilingual, spokesperson-style UGC; has a public API. Slightly heavier integration than Creatify but broader style range. Build the integration behind an interface (`generate_video(script, brand_assets, style)`) so you can swap or run both without rewriting the agent. Expect this to be your slowest and most expensive piece per generation — don't block the MVP demo on it.

Analytics & Learning Agent Ingests reported campaign results. Writes a retrospective. Embeds it into the business's memory store. On every new campaign, retrieves relevant memories into Research/Strategy Agent context. Critically: this agent should also be able to **push**, not just respond — a dashboard notification like "Your last 3 email subject lines with a number in them outperformed the rest by 40% — want the next batch to follow that pattern?" is the actual product moat here. A passive dashboard nobody checks isn't the same feature.

Distribution Agent — Phase 2, not MVP Publishing/scheduling to actual channels. Don't build this until the generation + learning loop is proven; it's OAuth and app-review overhead that doesn't validate anything about whether the campaigns themselves are good.

5. Agent chat (the “communicate with agents” requirement)

Each campaign gets one conversation thread per agent type (`campaign_id + agent_type` → `conversation`). When the user opens, say, the Email Agent’s chat and says “make the second email shorter and less formal,” that message is routed by the Orchestrator to the Email Agent, which has tool access to read and update that campaign’s email `content_asset` directly — the chat isn’t just talk, it edits the real underlying content via tool calls. Persist every agent conversation; it’s a second, richer source of “what the user actually wanted” for the Learning Agent to draw on later, beyond just the results numbers.

6. Non-negotiable build discipline (carried over from the original plan — don’t drop this in the pivot)

- **Audit before building.** Before writing code for any agent, state what already exists, what you’re changing, and wait for confirmation on anything structural.
 - **JSON output contracts, validated.** Every agent call returns schema-validated JSON. Retry once on validation failure, then fail loud with a stored reason — never fail silent.
 - **Log every agent action and every tool call it makes**, especially ones that edit content — this is how you debug “why did the Email Agent do that.”
 - **Verify before moving to the next piece.** Read actual agent output yourself. Generic, interchangeable-across-businesses output is a prompt problem — fix it before building on top of it.
 - **One reusable agent runtime, not seven bespoke ones** (restated because it’s the thing most likely to get skipped under time pressure).
-

7. What NOT to build yet (guardrails against scope creep)

- Full autonomous multi-agent negotiation/planning between agents — the Orchestrator calling specialists in a fixed, sensible order is enough for now. True agent-to-agent negotiation is a Phase 3+ idea, not an MVP requirement.
- Auto-publishing to ad platforms (Phase 2, per above).

- Video generation at MVP demo time if it's blocking everything else — ship the campaign as scripts + a "Generate Video" button that can lag behind the rest of the launch.
 - Seven separate codebases for seven agents. See Section 4.
-

8. Definition of "on track"

You're building the right thing if: a real website URL in → a complete, editable, multi-channel campaign out → the user can open at least one agent's chat and get a change that actually applies → a second campaign for the same business visibly reflects something learned from the first. If any of those four breaks, stop and fix it before adding another agent or another channel.